

AI FOR FIREFIGHTERS

by

Team Mango Tango

Cordelia Ludden, Yi Gu, Sophia Carney, Aryaa Modi, and Davae Lynch

Final Report

Capstone Project

Boston Fire Department, Karen Panetta

3 May 2026

Project Description:

Problem Background:

A firefighter commander has no way of seeing where firefighters are in a building during an emergency and it is difficult to determine their location based on verbal communication alone. Dr. Panetta, our sponsor, noted one particular event during a fire emergency in a building built on a pier over water. One of the firefighters radioed that he was trapped, but he could not provide the exact location. He started to dig himself out with his axe, but unfortunately died of smoke inhalation. If the rescue team had known his location, they could have helped break him free. This is the biggest issue that we are attempting to solve. Another problem is that in highly smoky environments, it is hard to see where the exits are in a building. Exits can be either windows or doors. One proposal was that a firefighter commander could use building blueprints to navigate firefighters during an emergency. But, it would not be feasible to find the blueprints of a given building in time. Even if they could, they might not be up-to-date. Therefore, we also wanted to find a way to let the firefighter commander know where doors and windows are located inside the building during a rescue.

Sponsors:

Our sponsor is Dr. Karen Panetta, professor of ECE at Tufts University. Her customer for this project is the Boston Fire Department. We also were majorly helped by Aileen Sengupta, who is a postdoctoral research scholar in Dr. Panetta's Vision and Sensing lab. She guided us on which LiDAR and IMU to get and gave us advice on how to use them and design code. Dr. Panetta provided us with background context and an overview of the project. She prompted us to use LiDAR in our project because it will help to see through smoke during a fire emergency.

Problem Statement:

The problem that we are trying to solve is using LiDAR to map a building to create a comprehensive dashboard that displays walls, windows, and doors. This dashboard will be used by a fire commander to help make safety decisions while firefighters respond to an emergency. We worked on this problem this year by focusing on getting the LiDAR to output x, y, z coordinates from a single frame and cleaning this data which would be piped into a machine learning model. This model takes in point cloud data and predicts which points belong to walls, windows, doors etc. The model also outputs the overall accuracy of its predictions.

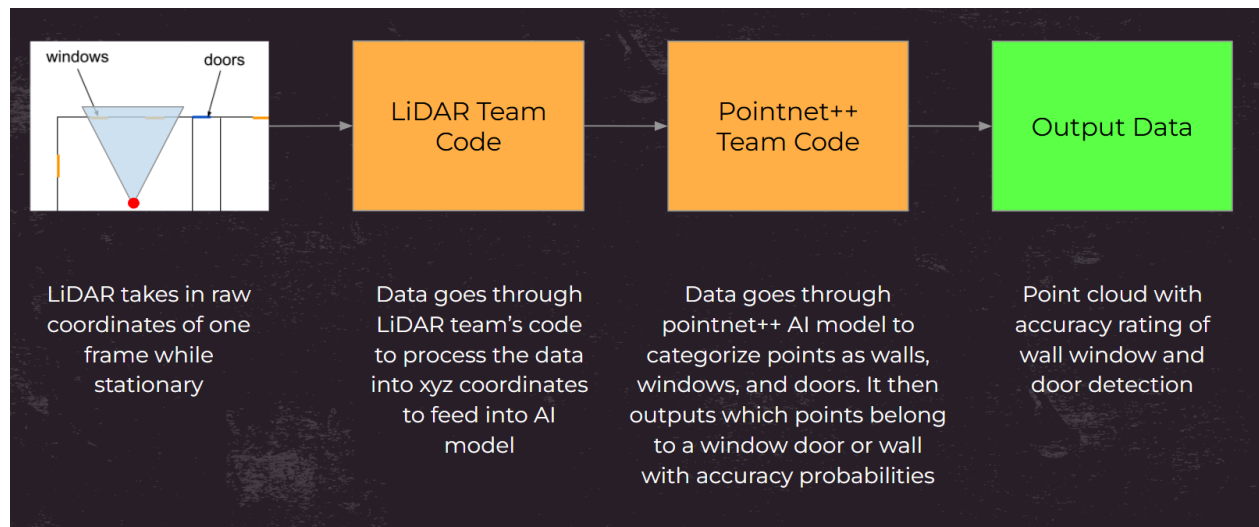
Project Requirements:

Our requirements include both getting the physical device work and the AI system to work. First, we need to get the LiDAR device to work, so it can collect and process point cloud data. Then

we need to build an AI model that takes in the point cloud data and does semantic segmentation on each point, that is to identify whether each point belongs to walls, windows, doors, or other indoor structures. Finally, we need to make sure the device is practical for the firefighters to use in real life. So we need to make the device wearable and resistant to water and heat.

Design Choices:

MVP:

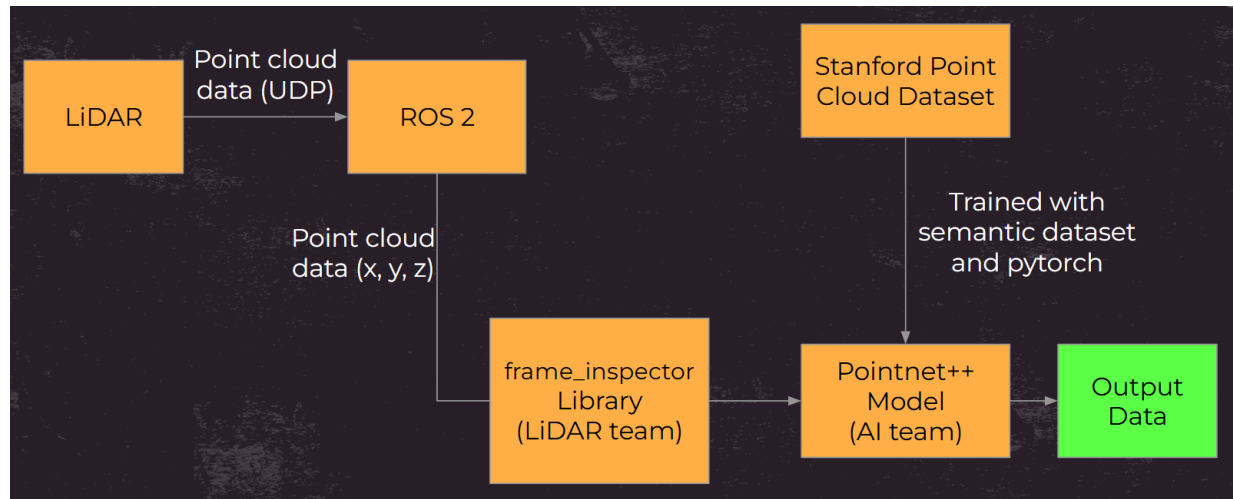


The data pipeline will take in Lidar point cloud data of one frame, and then be preprocessed and passed into the PointNet++ model. For each block of the point cloud data, the trained model will be able to identify it as a wall, window, or door.

Interface Design:

Our deliverable is being handed off to other computer scientists. Therefore the way to interact with it is through plugging in the LiDAR and running our code as follows. Full instructions for this and everything related to our deliverable and MVP can be found at our public github page's readme: <https://github.com/C3PO3/fightingFires>

System Architecture:



The LiDAR takes in positional data and transmits it in the form of UDP packets to our windows laptop running ROS 2, ROS 2 translates this into x, y, z coordinates which are pre-processed by our frame_inspector library which is attached to ROS 2. This data can be piped into our AI program (which uses a pointnet++ model and then outputs the resulting predictions about whether points belong to walls, windows, or doors. The AI model is trained at first by taking in data from the Stanford Pointcloud Dataset and uses pytorch for training the model. After training the model outputs its overall accuracy rating by testing itself against a portion of the data reserved for testing purposes.

Tools and Technologies Used:

Hardware used:

- Our LiDAR was: RoboSense E1R (forward-looking, solid-state “digital” LiDAR)
- VN-100 IMU - used for tracking location of the LiDAR needed for processing multiple frames at once
- Lab computer with a terabyte of data for storing Stanford dataset used for training AI model

Software Used:

- Ubuntu
- ROS 2 - Humble version
 - rslidar_sdk
- Docker (in preliminary work not final product)
- Pytorch
- [Stanford Pointcloud Database](#)

Acceptance Tests:

Test 1: Device must be able to accurately map a building

- Test Result: Fail
- Reason: We did not have the ability to aggregate multiple frames at a time, also our AI model's detection accuracy was not high enough. In other words this acceptance test relied on two other acceptance tests which did not pass.
- How to improve our result: We would need to pass test 4 and test 5. The way to fix test 4 is to integrate the IMU so we can look at multiple frames at a single time to map out the whole building instead of a single dataframe. The way to fix test 5 is to try training the model with a more balanced dataset or using a machine with GPUs instead of CPUs to train the model.

Test 2: Device must be wearable and non intrusive to the firefighters normal activities

- Test Result: Pass
- Reason: We are able to strap the LiDAR to a harness which would be wearable by the firefighter. When purchasing our LiDAR we got a forward facing LiDAR to aid in this objective.

Test 3: LiDAR must be able to transmit frame by frame data into AI model

- Test Result: Pass
- Reason: We added a package called [frame inspector](#) which works with ROS 2 to turn UDP packets transmitted by the LiDAR into xyz coordinates and pipes them out so they can be fed into other code. This data is sent into our AI program which can process these xyz coordinates and make predictions based on this data.

Test 4: LiDAR can process multiple frames in relation to one another

- Test Result: Fail
- Reason: We cannot process multiple frames at a time since we would need to aggregate these together based on the firefighters movements. Since the LiDAR does not keep track of its own movements we would need to attach something to the LiDAR which is capable of tracking how it moves so these different frames could be placed in relation to one another.
- How to improve our result: This can be fixed by integrating the IMU with the LiDAR so that the IMU takes in positioning data, we use clock sync with the LiDAR and IMU, and then use the offset of the LiDAR's motion to place the different point clouds in relation to one another, so the movement of the LiDAR does not affect our ability to process multiple frames.

Test 5: The AI model must be able to accurately perform wall, window, and door detection. It should be tested with an unseen environment and avoid overestimating performance

- Test Result: Fail
- Reason: We got a lower than expected accuracy score from our program. We got an overall program accuracy of 62% and then a per class accuracy score of 37%. The reason for this might have been due to the fact that the Stanford dataset that we were working with had a class imbalance within it which we didn't account for. This class imbalance was clear in our results as the dominant classes were being predicted more and had a higher accuracy than others.
- How to improve our result: One suggestion to improve the result is to use a dataset that is more balanced to train the model. Another possible solution would be to resampling, such as increasing the number of times the minority classes are represented, to remedy the imbalance in the dataset.

Project Management

Organization

At first, we were all working together on each step of the project. As the project went on, we found that a natural division would be to create two subteams: the LiDAR team and the AI Modeling team. Two members, Sophia and Cordelia, were on the LiDAR team and Yi, Aryaa, and Davae were on the AI Modeling team. The LiDAR team worked on setting up the LiDAR and ensuring that frames and point cloud data can be extracted from it. The AI Modeling team worked on improving the accuracy of detecting windows, doors, and walls using a PointNet++ model.

Timeline

Sprint 1:

- Order LiDAR
- Get access to Stanford Pointcloud Database
- Research wall mapping techniques for point cloud datasets (looking into [pcl](#))

Sprint 2:

- Received LiDAR and downloaded ROS2 and SDK
- Used pcl datasets to attempt wall detection using RANSAC models

Sprint 3:

- Split into two subteams for LiDAR and AI model

- LiDAR team fixed errors with connecting the LiDAR and got packets to successfully be received over the network
- The AI team realized the RANSAC model does not work on windows and doors, and pivoted to other AI models. Researched pointnet++ model and decided to attempt using that. Requested laptop from the lab to run the model.

Sprint 4:

- The LiDAR team wrote a package compatible with the LiDAR's SDK to process point cloud frame data into xyz coordinates, transmitted the data to the old program that ran the RANSAC models as a sanity test and was successful. Discovered issues with transmitting point cloud data when LiDAR is moving and discovered the LiDAR's inability to track its own motion
- The AI team worked on writing code to preprocess the original mesh-level data from Stanford dataset to point level data compatible with the data getting from LiDAR. Then partitioned the data into smaller blocks so it can be taken by the PointNet++ model for training.

Sprint 5:

- The LiDAR team worked with our sponsor to find strategies to deal with processing multiple frames at once without location data. They tried looking at the SLAM algorithm, but it relies on a 360 LiDAR, then tried using the NDT algorithm provided by the sponsor, but this did not work as well. The team settled on ordering an IMU to get location based data from the LiDAR instead.
- The AI team built Pointnet++ model using Pytorch and trained the model. They had the model output accuracy scores.

Sprint 6:

- The LiDAR team looked through IMU documentation, figured out how to use it and how to connect it to the LiDAR. Got the IMU working, but did not have time to connect it.
- The AI team tested the model with different metrics to evaluate the performance and worked on improving the model's performance.

Sprint 7:

- The LiDAR team went over steps to duplicate our work thus far and documented it, making a public github repo with instructions to be added to. Continued to work with integrating the IMU and understanding the metrics it used.
- The AI team continued working on improving the model's accuracy by adjusting metrics. Made a github repo to start documenting their work as well.

Sprint 8:

- Both teams worked together to make comprehensive instructions on how to duplicate all of our work and be clear about the next steps for the project. Have all important

documents stored in a public github repository not in google, so that other teams can benefit from our work and do not have to start from scratch.

Ethics and Social Impact

The ethics and social impact of this project mainly focus on safety and reliability of our system. Since the system is built to help firefighters during emergencies, inaccurate predictions may lead to serious consequences. For example, if the system incorrectly identifies a spot as a door or a window that actually does not exist, the commander may assume there is a possible exit or access point, thus leading to wrong decisions. Therefore in this system, false positives can be more harmful than false negatives. This means it should prioritize avoiding making incorrect detections. Since the system may be used by the firefighters in life-threatening situations, it should be strictly tested before handing over.

Learning Reflections

We learned a whole lot throughout this project because of the challenges we encountered. The main challenge is that we realized due to time and technology constraints that we had to change our MVP and lower the scope on our project. Initially, we wanted to integrate thermal and RGB cameras with the LiDAR and create a dashboard that would also show that data. However, we realized this was not possible so we focused on ensuring we got proper data output from the LiDAR and created an AI program that would take in the data and make proper predictions. Through changing our MVP many times, we learned how to be adaptable and to change our goals when it seems we might not be able to complete them. We also learned the importance of splitting tasks among a team because it helped us to complete more tasks in the project. For example, if one team is blocked by something that hinders their ability to continue with their task, they can help out the other team.

For the next steps of this project, we suggest the following:

- Integrating the LiDAR with IMU so that a point cloud of an entire building can be created from many LiDAR frames.
- Piping the point cloud data from the LiDAR into the AI model to see if doors, windows, and walls are accurately being detected.
- Improving the results of the AI model by doing more training on different scenes.
- Testing the AI model with LiDAR data that is actually representative of fire conditions such as data from smoky environments.
- Creating a dashboard that shows a birds-eye view map of a building and highlights where the doors and windows are, as well as where the person holding the device is on the map.

References:

- Stanford Pointcloud Dataset - <https://sdss.redivis.com/datasets/f304-a3vhsvacaf>
- Point cloud library - <https://pointclouds.org/>

Appendices:

- Our github - <https://github.com/C3PO3/fightingFires>